

ALIRIN

Aplikasi Pelaporan & Prediksi Drainase Mikro

Kota Bandar Lampung - Smart City

SKRIP PRESENTASI LENGKAP (per Halaman)

Pembagian tugas:

KHAIRI

Pembuka, Arsitektur, Splash/Onboarding, Login & Beranda

REVALDO

Lapor, Peta Risiko & Status Laporan

NAUFAL

Navigasi/Peran, Sisi Staff, Tentang, Web & Penutup

PEMBUKA - KHAIRI: Produk, Tech Stack & Arsitektur

Khairi:

Assalamualaikum / selamat pagi Bapak/Ibu dosen. Perkenalkan, kami tim ALIRIN. Kami akan mempresentasikan aplikasi mobile bernama ALIRIN: sistem pelaporan sekaligus prediksi drainase mikro untuk Kota Bandar Lampung.

Latar belakangnya: genangan dan banjir kecil di kota sering terjadi bukan hanya karena hujan, tapi karena drainase (got) mampet, tertutup sampah, atau tidak terawat. Penanganannya selama ini reaktif - hujan turun, banjir, baru warga lapor, baru ditangani. ALIRIN mengubah pola ini jadi PREVENTIF: laporan warga dan data cuaca diolah menjadi prioritas aksi dan prediksi risiko.

Ada tiga peran pengguna dalam sistem ini:

- **Warga** melapor drainase bermasalah, melihat peta risiko, dan memantau status.
- **Staff** memvalidasi dan menindaklanjuti laporan - semuanya di dalam aplikasi.
- **Admin** mengelola data lewat website; aplikasi mobile mengarahkan ke web.

Dari sisi teknologi, aplikasi dibangun native Android dengan Kotlin dan Jetpack Compose untuk antarmukanya. Kami memakai 3 sumber data dari internet:

- **BMKG** API resmi pemerintah untuk prakiraan cuaca per kelurahan.
- **GROQ AI** model AI (Llama) untuk memprediksi kondisi udara, suhu, curah hujan, debit air.
- **dummyjson** layanan autentikasi untuk login staff dan admin.

Pola arsitektur yang kami pakai adalah MVVM (Model-View-ViewModel) dengan pemisahan lapisan. Setiap lapisan punya tugas sendiri dan tidak saling mencampuri:

```

UI (Jetpack Compose)  -> hanya menampilkan, tidak tahu soal API
    | StateFlow (data mengalir otomatis)
ViewModel             -> menyimpan & menyiapkan state untuk UI
    |
Repository            -> memproses & menggabungkan data
    |
Network (Retrofit)    -> Service / ApiClient / DTO -> API
  
```

Keuntungan pemisahan ini: mudah dirawat dan diuji. Mengganti tampilan tidak mengubah logika; mengganti sumber data tidak mengubah tampilan. Struktur foldernya mengikuti tanggung jawab tadi:

```

data/network/         -> service/ (endpoint), dto/ (model JSON), ApiClient
data/repository/      -> Auth, Weather, Prediction, Report
data/local/           -> DataStore (menyimpan token login)
feature/              -> layar per fitur (auth, beranda, lapor, peta, staff)
ui/theme/             -> warna, font, ukuran (satu sumber design system)
ui/components/        -> komponen dipakai ulang (Pill, Avatar, Button, Card)
nav/                  -> pengatur navigasi & peran pengguna
  
```

Sekarang saya lanjut menjelaskan halaman-halaman awal aplikasi.

BAGIAN 1 - KHAIRI: Splash, Onboarding, Login & Beranda

> Splash Screen

file: *feature/auth/SplashScreen.kt*

Khairi:

Saat aplikasi dibuka, yang pertama muncul adalah Splash Screen: logo tetesan air ALIRIN dengan latar hijau-teal selama sekitar 1,8 detik. Ini juga mencegah layar putih berkedip saat aplikasi dingin (cold start) karena kami pakai SplashScreen API resmi Android.

feature/auth/SplashScreen.kt

```
LaunchedEffect(Unit) {
    delay(1800)          // tahan 1.8 detik
    onDone()             // lanjut ke layar berikutnya
}
```

> Onboarding (3 slide)

file: *feature/auth/OnboardingScreen.kt*

Setelah splash, muncul Onboarding berisi 3 slide pengenalan: (1) Lapor Cepat 10 detik, (2) Verifikasi gotong-royong, (3) Prediksi cuaca BMKG. Slide bisa digeser. Statusnya disimpan permanen di DataStore, jadi onboarding hanya muncul sekali - di buka berikutnya langsung lewat.

nav/AlirinNavHost.kt

```
!onboardingDone -> OnboardingScreen(onDone = { authVm.finishOnboarding() })
// finishOnboarding() menyimpan flag ke DataStore
```

[peragakan: Splash -> geser Onboarding -> Mulai pakai ALIRIN]

> Welcome Screen

file: *feature/auth/WelcomeScreen.kt*

Halaman Welcome adalah pintu masuk. Warga bisa langsung menekan 'Mulai sebagai Warga' tanpa perlu daftar atau login - sesuai prinsip pelaporan publik yang cepat. Sementara petugas menekan tautan kecil 'Masuk sebagai Staff' di bawahnya. Pembedaan ini disengaja: warga harus semudah mungkin, akses staff sedikit tersembunyi.

nav/AlirinNavHost.kt

```
WelcomeScreen(
    onWarga = { authVm.chooseAnonymous() }, // masuk anonim
    onStaff = { showStaffLogin = true },    // buka form login
)
```

> Login Staff

file: *feature/auth/LoginScreen.kt + LoginContent.kt*

Untuk login, kami pisahkan menjadi dua file sebagai contoh pola pemisahan: LoginScreen menangani logika dan state, sedangkan LoginContent murni tampilan - hanya menerima data dan aksi lewat parameter. Dengan begitu tampilannya bisa dipratinjau dan diuji terpisah dari logika.

Proses login memanggil API lewat AuthViewModel. State login (loading, sukses, gagal) dibungkus rapi sehingga UI tinggal bereaksi.

feature/ViewModels.kt (AuthViewModel)

```
fun login(username, password) {
    ui.value = Submitting
    viewModelScope.launch {
```

```
runCatching { repository.login(username, password) }  
    .onSuccess { ui.value = Ok(it) }  
    .onFailure { ui.value = Failed(it.message) }  
}  
}
```

Token hasil login disimpan di DataStore (penyimpanan lokal aman), dan dipakai otomatis untuk permintaan berikutnya. Username menentukan peran: emilys=staff, michaelw=admin.

> Beranda + Kartu Prediksi AI

file: feature/beranda/BerandaScreen.kt

Inilah halaman utama warga. Di atas ada sapaan + nama kecamatan terpilih, lalu grid menu: Lapor Cepat, Lapor Lengkap, Peta Risiko, dan Status. Di bawahnya ada strip cuaca BMKG, dan komponen andalan kami: Kartu Prediksi AI.

Kartu prediksi menampilkan ringkasan, kondisi udara, suhu, curah hujan 3 jam ke depan, estimasi debit air drainase, plus rekomendasi tindakan - semuanya hasil olahan AI dari data BMKG. Datanya reaktif memakai StateFlow, jadi begitu pengguna ganti kelurahan, kartunya otomatis menghitung ulang.

feature/beranda/BerandaScreen.kt

```
val prediction by vm.prediction.collectAsStateWithLifecycle()  
// AiForecast: kondisiUdara, suhuCelsius, curahHujanMm,  
//            debitAirMs, ringkasan, rekomendasi[]  
if (p == null) PredictionCardSkeleton() // loading  
else PredictionCardContent(model = p)   // tampilkan hasil
```

Detail teknis bagaimana AI memprosesnya ada di slide khusus nanti. Saya serahkan ke Revaldo untuk halaman Lapor, Peta, dan Status.

BAGIAN 2 - REVALDO: Lapor, Peta Risiko & Status

Revaldo:

Terima kasih Khairi. Saya menjelaskan tiga aktivitas inti warga: membuat laporan, melihat peta risiko, dan memantau status laporan.

> Alur Lapor - dibuka sebagai drawer

file: feature/lapor/LaporFlowScreen.kt

Saat tombol Lapor (tombol + di menu bawah) ditekan, form lapor muncul sebagai drawer yang naik dari bawah layar - bukan pindah halaman penuh - sehingga terasa ringan. Alur isinya 4 langkah berurutan yang dikelola satu state machine.

feature/lapor/LaporFlowScreen.kt

```
when (step) {
    Choice -> ChoiceStep(...) // pilih Cepat / Lengkap
    Lokasi -> LokasiStep(...) // GPS / pin lokasi
    Detail -> DetailStep(...) // kategori + keparahan
    Foto -> FotoStep(...) // foto (Lengkap saja)
    Success -> SuccessScreen(...) // nomor laporan keluar
}
```

> Langkah 1-2: Pilih Mode & Lokasi

file: feature/lapor/ChoiceStep.kt, LokasiStep.kt

Langkah pertama memilih Lapor Cepat (tanpa foto, 10 detik) atau Lapor Lengkap (dengan foto, skor lebih tinggi). Langkah kedua menentukan lokasi: pengguna menekan 'Gunakan lokasi saya' yang meminta izin GPS lalu mengisi koordinat otomatis; kecamatan dan kelurahan mengikuti pilihan pengguna.

feature/lapor/LokasiStep.kt

```
locVm.fetchOnce { loc ->
    if (loc != null)
        onUpdate(form.copy(lat=loc.lat, lng=loc.lng,
                           accuracyMeters=loc.accuracyMeters))
}
```

> Langkah 3-4: Detail & Foto (watermark)

file: feature/lapor/DetailStep.kt, FotoStep.kt

Langkah ketiga memilih kategori masalah (6 pilihan) dan tingkat keparahan (4 tingkat). Langkah keempat (hanya untuk Lapor Lengkap) menambah foto. Foto dari kamera otomatis diberi watermark berisi koordinat dan waktu - supaya tidak bisa dipalsukan dengan foto lama. Foto dari galeri ditandai 'tidak diverifikasi lokasi'.

data/PhotoStore.kt

```
PhotoStore.saveCameraBitmap(ctx, bitmap, lat, lng)
// menambahkan teks 'lat, lng + waktu' di pojok foto
```

> Kirim Laporan & Success

file: feature/lapor/SuccessScreen.kt

Saat dikirim, ViewModel menghitung skor risiko dari tingkat keparahan dan mode, lalu menyimpan laporan ke ReportRepository. Pengguna mendapat nomor laporan unik (ALR-2026-xxxx) dan ringkasan. Laporan baru langsung muncul di daftar status.

feature/ViewModels.kt (LaporViewModel)

```
fun submit() {
    val result = repository.submitReport(form.value, mode.value)
    submitted.value = result.code // ALR-2026-xxxx
}
```

[peragakan: *Lapor Lengkap -> lokasi -> kategori -> foto -> Kirim -> nomor laporan*]

> Peta Risiko

file: *feature/peta/PetaScreen.kt + OsmMapView.kt*

Peta memakai osmdroid dengan peta OpenStreetMap - gratis, tanpa API key, tanpa biaya. Di atas peta ada kartu 'Area kamu' yang dihitung dari kelurahan terpilih dan titik di sekitarnya. Setiap titik rawan jadi marker berwarna sesuai level risiko. Ada kolom pencarian, filter sumber data, tombol zoom, dan legenda warna.

Marker digambar langsung di peta sehingga ikut bergeser saat peta digeser/di-zoom. Tap marker membuka detail di bottom sheet: kategori, level, skor, status, dan tombol 'Tambah laporan di sini'.

feature/peta/PetaScreen.kt

```
OsmMapView(
    hotspots = filtered, // hasil filter + pencarian
    selectedId = selected?.id,
    onHotspotTap = { selected = it },
    onMapReady = { mapRef = it }, // referensi utk tombol zoom
)
```

> Status Laporan (daftar)

file: *feature/status/StatusListScreen.kt*

Halaman ini menampilkan laporan milik pengguna. Ada pencarian (berdasarkan nomor, kategori, atau kelurahan) dan filter Semua / Aktif / Selesai yang langsung menyaring daftar. Tiap kartu menampilkan nomor, kategori, lokasi, badge status, dan progres mini.

feature/status/StatusListScreen.kt

```
val filtered = reports
    .filter { sesuaiFilter(it) } // Semua/Aktif/Selesai
    .filter { it.code.contains(query) || it.category... }
```

> Detail Status (timeline)

file: *feature/status/StatusDetailScreen.kt*

Detail laporan menampilkan informasi lengkap dan timeline 6 tahap: Menunggu -> Diverifikasi -> Dijadwalkan -> Ditangani -> Selesai (atau Ditolak). Setiap tahap menampilkan waktu dan catatan dari petugas. Ada juga tombol Bagikan yang memakai share sheet Android. Saya lanjut ke Naufal untuk sisi staff.

BAGIAN 3 - NAUFAL: Navigasi/Peran, Sisi Staff, Tentang & Web

Naufal:

Terima kasih Revaldo. Saya mulai dari bagaimana aplikasi menentukan tampilan berdasarkan peran pengguna, lalu menjelaskan seluruh halaman sisi staff.

> Navigasi & Pembagian Peran

file: nav/AlirinNavHost.kt

Pusat navigasi menentukan layar berdasarkan status: belum splash, belum onboarding, sudah login (peran apa), atau anonim. Admin langsung diarahkan ke halaman web; warga dan staff masuk ke aplikasi utama dengan menu bawah yang berbeda.

nav/AlirinNavHost.kt

```
when {
    !splashDone        -> SplashScreen()
    !onboardingDone    -> OnboardingScreen()
    role == Admin      -> AdminWallScreen()    // arahkan ke web
    login / anonim     -> MainShell()          // warga / staff
    else               -> WelcomeScreen()
}
```

Transisi antar layar dibuat halus bergaya iOS (slide), dan menu bawah muncul/hilang dengan animasi mulus mengikuti lifecycle.

> Admin Wall

file: feature/auth/AdminWallScreen.kt

Jika yang login adalah admin, aplikasi tidak menampilkan dashboard di HP, melainkan halaman yang menjelaskan bahwa pengelolaan admin dilakukan lewat website. Ini sesuai pembagian: mobile fokus untuk warga dan staff lapangan.

> Inbox Verifikasi (Staff)

file: feature/staff/PersetujuanListScreen.kt

Setelah login sebagai staff, menu bawah berganti - muncul tab Verifikasi. Halaman ini berisi kartu statistik (jumlah menunggu, jumlah kritis, rata-rata respon), filter (Semua/Kritis/Dengan foto/Tanpa foto), dan daftar laporan yang menunggu validasi. Laporan kritis ditandai border merah agar menonjol.

> Detail & Aksi Validasi

file: feature/staff/PersetujuanDetailScreen.kt

Di detail, staff melihat foto bukti, deskripsi, dan 'sinyal validasi': berapa warga yang melapor, ada/tidaknya hotspot historis, dan skor risiko sistem. Aksinya lewat drawer: Verifikasi (beri catatan), Tolak (pilih alasan), atau Jadwalkan (pilih tim dan waktu). Ada juga tombol Navigasi yang membuka aplikasi peta ke lokasi.

feature/ViewModels.kt (StaffViewModel)

```
fun transition(reportId, newStatus, note) {
    repository.updateReportStatus(reportId, newStatus,
        note, actorLabel)    // status + catatan masuk riwayat
}
```

Karena memakai StateFlow, perubahan status oleh staff ini LANGSUNG terlihat di halaman Status milik warga - tanpa perlu refresh. Inilah inti alur end-to-end aplikasi.

[peragaan: login emilys/emilyspass -> Verifikasi -> pilih laporan -> Verifikasi -> sukses]

> Tindak Lanjut, Statistik, Profil

file: feature/staff/StaffTindakLanjutScreen.kt, StaffStatistikScreen.kt, StaffProfilScreen.kt

Tab Lanjut menampilkan pekerjaan tim (berjalan/selesai) dengan tombol Tutup untuk menyelesaikan. Tab Stats berisi grafik 14 hari, pecahan kategori, dan top kelurahan - dihitung otomatis dari data laporan, bukan angka tetap. Tab Profil berisi info staff dan tombol 'Ganti peran' untuk keluar dan kembali ke pemilihan peran.

feature/staff/StaffStatistikScreen.kt

```
// pecahan kategori dihitung dari daftar laporan, bukan hardcode
val categoryBreakdown = reports
    .groupBy { it.category }.eachCount()
    .entries.sortedByDescending { it.value }
```

> Tentang & ALIRIN Web

file: feature/tentang/TentangScreen.kt, feature/web/WebViewScreen.kt

Halaman Tentang menjelaskan cara kerja sistem, empat sumber data, dan kebijakan privasi (anonim per perangkat). Ada tautan 'Buka ALIRIN Web' yang membuka situs di dalam aplikasi menggunakan WebView, lengkap dengan indikator loading dan tombol kembali.

Naufal:

Sebagai penutup, rencana pengembangan kami: backend real-time agar laporan tersinkron lintas perangkat, integrasi sensor IoT ketinggian air, dan notifikasi dini cuaca. Demikian presentasi dari tim kami. Terima kasih, kami buka sesi tanya jawab.

LAMPIRAN - Detail Teknis (untuk pertanyaan mendalam)

> Lapisan Network - 3 file

file: data/network/

Koneksi API kami pisah jadi tiga tanggung jawab dengan Retrofit:

- **Service** interface daftar endpoint - APA yang dipanggil.
- **ApiClient** atur header, token, timeout, base URL - BAGAIMANA memanggil.
- **DTO** data class @Serializable - bentuk JSON yang diterima.

data/network/service/BmkgService.kt

```
interface BmkgService {
    @GET("publik/prakiraan-cuaca")
    suspend fun forecast(@Query("adm4") adm4: String)
        : BmkgForecastResponse
}
```

> Repository - menggabungkan BMKG + AI

file: data/repository/PredictionRepository.kt

Repository inilah otak penggabungan. Data cuaca BMKG disusun jadi prompt, dikirim ke GROQ, lalu jawabannya (JSON) diubah jadi objek AiForecast.

```
val res = api.groqService.chat(ChatCompletionRequest(
    messages = listOf(
        ChatMessage("system", "prediksi cuaca + debit air, JSON..."),
        ChatMessage("user", "curah hujan: $precip mm, suhu: $temp C"),
    ))
return json.decodeFromString<AiForecast>(rawJson)
```

Pengaman: jika API key tidak ada atau internet mati, sistem otomatis pakai perhitungan rule-based - aplikasi tetap menampilkan prediksi, tidak error.

```
if (!api.groqConfigured) return fallback(forecast)
return runCatching { callGroq(...) }
    .getOrElse { fallback(forecast) }
```

TANYA-JAWAB - Bocoran Pertanyaan Dosen + Jawaban

T: Datanya dari mana?

J: Cuaca: API resmi BMKG. Prediksi: AI GROQ (model Llama). Login: dummyjson. Data laporan saat ini contoh (dummy) di memori - arsitektur sudah siap dihubungkan ke backend nyata tanpa mengubah tampilan.

T: Cara kerja prediksi AI-nya bagaimana?

J: Data cuaca BMKG (curah hujan, suhu, kelembaban 3 jam ke depan) disusun jadi prompt, dikirim ke model AI GROQ. AI mengembalikan JSON berisi kondisi udara, suhu, curah hujan, estimasi debit air drainase, dan rekomendasi tindakan.

T: Kalau tidak ada internet / API gagal?

J: Ada perhitungan cadangan (rule-based). Aplikasi tetap menampilkan prediksi, tidak error.

T: Cara mengambil data dari API gimana?

J: Memakai Retrofit, dipisah 3 file: Service (endpoint), ApiClient (header/token/timeout), DTO (bentuk JSON). Repository memproses dan menggabungkan hasilnya, lalu dialirkan ke UI lewat ViewModel.

T: Kenapa memakai MVVM?

J: Memisahkan tampilan, logika, dan data. Mudah dirawat, diuji, dan diganti per bagian tanpa merusak bagian lain.

T: Kok datanya update otomatis?

J: Memakai StateFlow. UI 'berlangganan' data; ketika data berubah (mis. staff memvalidasi), tampilan otomatis tergambar ulang tanpa refresh manual.

T: Peta pakai apa? Berbayar?

J: osmdroid dengan OpenStreetMap - gratis, tanpa API key, tanpa biaya.

T: Bagaimana keamanan API key AI?

J: Disimpan di local.properties (di-gitignore), dibaca lewat BuildConfig. Tidak ikut ter-upload ke repository GitHub.

T: Bedanya Lapor Cepat dan Lengkap?

J: Cepat: tanpa foto, ringan, skor bukti lebih rendah. Lengkap: dengan foto ber-watermark, skor lebih tinggi sehingga lebih diprioritaskan petugas.

T: Apa itu verifikasi gotong-royong?

J: Jika 3 warga atau lebih melapor titik yang sama dalam radius dan waktu berdekatan, laporan otomatis dianggap terverifikasi oleh warga.

T: Foto bisa dipalsukan tidak?

J: Foto dari kamera otomatis diberi watermark koordinat + waktu. Foto dari galeri ditandai 'tidak diverifikasi lokasi'.

T: Kenapa skor risiko bisa beda tiap laporan?

J: Dihitung dari tingkat keparahan dikali pengali mode (Lengkap lebih tinggi dari Cepat), menghasilkan level Normal/Waspada/Tinggi/Kritis.